



INFORMATICA

UNIVERSIDAD MAYOR DE SAN ANDRES



TALLER DE SISTEMAS

LUNES 18 DE MAYO – DOCUMENTACION

CARRERA DE INFORMATICA

Desarrollo de software e innovación tecnológica

Yherco Yhafar Espinoza Ticona

6854467 L.P.

FECHA: 18/05/2026

LA PAZ BOLIVIA



1. Descripción Técnica

El sistema centraliza las operaciones a través de un control estricto de accesos y perfiles de usuario. La seguridad y consistencia operativa se rigen mediante la tabla ROLES, la cual define dinámicamente los niveles de autorización en la plataforma, y la tabla USUARIOS, que implementa un almacenamiento seguro de credenciales utilizando encriptación criptográfica unidireccional (password_hash). El sistema maneja una máquina de estados para los usuarios (estado VARCHAR(10) DEFAULT 'activo' NOT NULL), restringiendo de forma automatizada mediante un CONSTRAINT chk_estado cualquier intento de inicio de sesión de cuentas marcadas como inactivas.

2. Flujo de Control en el Backend y Frontend

En el backend (pet_spa_backend), las peticiones entrantes atraviesan la capa de routes hacia los middlewares de autenticación, donde se interceptan y validan los tokens antes de otorgar acceso a los controllers. En el frontend (pet_spa), la carpeta lib/src/view/auth maneja los ciclos de vida de las sesiones y restringe visualmente las vistas mediante un sistema de rutas protegidas basadas en el rol.

3. Evidencias del Módulo

- **Lógica de Autenticación (Backend):** SEGURIDAD_NODEJS

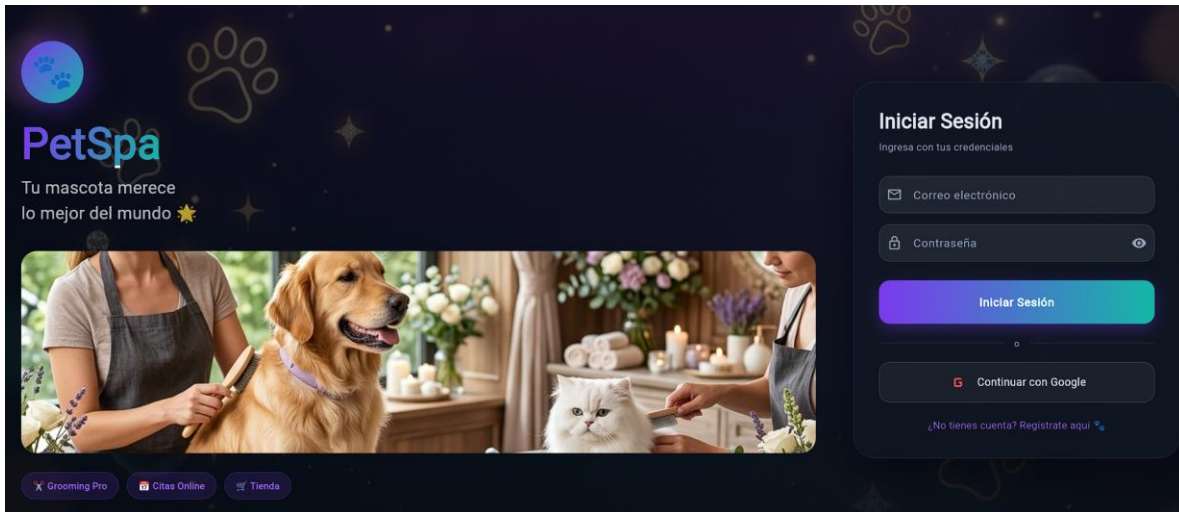
The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure for 'SPA_MASCOTAS'. The 'pet_spa_backend' folder is expanded, showing subfolders like 'logs', 'node_modules', 'sql', and 'src'. The 'src' folder is further expanded, showing 'auth', 'controllers', 'data', 'middlewares', 'routes', 'services', and 'utils'. The 'controllers' folder is selected, and 'auth.controller.js' is highlighted. On the right, the editor shows the code for 'auth.controller.js'. The code defines an 'authservice' module, a 'requestMeta' function, and two export functions: 'registroCliente' and 'registroAdmin'. The 'registroAdmin' function is currently selected.

```
auth.controller.js
1  const authservice = require('../services/auth.service');
2
3  const requestMeta = (req) => ({
4    ip: req.ip || req.headers['x-forwarded-for'] || req
5    browser: req.headers['user-agent'] || 'unknown',
6  });
7
8  exports.registroCliente = async (req, res) => {
9    try {
10     const result = await authservice.registerCliente(req);
11     if (result.success) {
12       return res.status(201).json(result);
13     }
14     return res.status(400).json({ message: result.message });
15   } catch (error) {
16     console.error(error);
17     return res.status(500).json({ message: 'Error interno en el servidor' });
18   }
19 };
20
21 exports.registroAdmin = async (req, res) => {
22   try {
23     const result = await authservice.registerAdmin(req.body);
24     if (result.success) {
25       return res.status(201).json(result);
26     }
27     return res.status(400).json({ message: result.message });
28   } catch (error) {
29     console.error(error);
30     return res.status(500).json({ message: 'Error interno en el servidor' });
31   }
32 };
33
34 module.exports = { registroCliente, registroAdmin };
```

The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure for 'SPA_MASCOTAS'. The 'pet_spa_backend' folder is expanded, showing subfolders like 'logs', 'node_modules', 'sql', and 'src'. The 'src' folder is further expanded, showing 'auth', 'controllers', 'data', 'middlewares', 'routes', 'services', and 'utils'. The 'routes' folder is selected, and 'auth.router.js' is highlighted. On the right, the editor shows the code for 'auth.router.js'. The code defines an 'express' module, a 'router' instance, and imports the controllers. It then defines several routes for client and admin registration, login, and user management. The routes are currently selected.

```
auth.router.js
1  const express = require('express');
2  const router = express.Router();
3  const authController = require('../controllers/auth.controller');
4  const empleadosController = require('../controllers/empleados.controller');
5  const clientesController = require('../controllers/clientes.controller');
6  const { authenticateToken, requireAdmin } = require('../middlewares/auth');
7
8  router.post('/registro_cliente', authController.registroCliente);
9  router.post('/registro_cliente_admin', authenticateToken, requireAdmin, authController.registroAdmin);
10 router.post('/registro_empleado', authenticateToken, requireAdmin, empleadosController.registroEmpleado);
11 router.post('/registro_admin', authController.registroAdmin);
12 router.post('/login', authController.login);
13 router.post('/login_google', authController.loginWithGoogle);
14 router.post('/verify_2fa', authController.verifyTwoFactor);
15 router.get('/activar/:token', authController.activarCuenta);
16
17 router.get('/empleados', authenticateToken, requireAdmin, empleadosController.listarEmpleados);
18 router.put('/empleados/:id', authenticateToken, requireAdmin, empleadosController.actualizarEmpleado);
19 router.delete('/empleados/:id', authenticateToken, requireAdmin, empleadosController.eliminarEmpleado);
20 router.get('/habilidades', authenticateToken, requireAdmin, empleadosController.listarHabilidades);
21
22 router.get('/clientes', authenticateToken, requireAdmin, clientesController.listarClientes);
23 router.put('/clientes/:id', authenticateToken, requireAdmin, clientesController.actualizarCliente);
24 router.delete('/clientes/:id', authenticateToken, requireAdmin, clientesController.eliminarCliente);
25
26 module.exports = router;
```

- Interfaz de Login (Frontend): VISTA_AUTENTICACION



- Diccionario de Datos (Roles y Usuarios): ESQUEMA_DDL_AUTH

```

8
9  SELECT * FROM ROLES;
10
11  INSERT INTO USUARIOS(id_rol, nombre, email, password_hash) VALUES
12  (1, 'Yherco Yhafar Espinoza Ticona', 'yhercobarco@gmail.com', '685
13  ;
14
15  SELECT * FROM USUARIOS;
16  SELECT * FROM CLIENTES;
17
18  UPDATE USUARIOS
19  SET estado = 'activo';

```

Data Output Messages Notifications

	id_rol [PK] integer	nombre character varying (256)	description text
4	4	ADMINISTRADOR	Acceso total al sistema
5	5	RECEPCION	Personal de recepción
6	6	GROOMER	Groomer o personal de servicio
7	7	CLIENTE	Dueño de mascota